OPTIMIZATION TECHNIQUES

GROUP PROJECT

Multi-Objective Optimization for a Real-World Problem

Group Size	4 students per group
Project Duration	8 weeks
Domain	Any domain — Computer / IT / AI / Healthcare / Engineering / Business / Agriculture / Energy / Transportation / Environment / etc.
Development Platform	GitHub or GitLab
Project Type	Real-world optimization problem with justified use of multi-objective optimization

Quick Start: What Every Group Must Do

Read this page first. The sections mentioned in the table explain each step in detail. Deadlines and the method of submission for every step will be announced by the teacher.

Step	What to do
1	Form a group of exactly four students.
2	Find and submit THREE genuinely different topic proposals (Sections 5 and 6). Do not choose anything from the sample list in Section 4A.
3	Wait for the professor to approve one of your topics. It may be modified, or you may be asked to propose another.
4	Immediately after approval: create a PRIVATE repository, add the professor and your three group members, and submit the repository-link document (Section 6A).
5	Weeks 1–8: follow the weekly plan (Section 13). All four members work in parallel and commit every week (Sections 14 and 15).
6	Keep code, folders and README.md in the format of Section 12A so that anyone can download and run your project. This is checked every week.
7	Final submission: code, README, report (Section 17), presentation/demo and viva.

Professor's GitHub/GitLab username (for repository access): [to be announced by the professor].

1. Project Objective

The objective of this project is to identify a meaningful real-world decision problem, formulate it as an optimization problem, identify the competing objectives and analyze them with a multi-objective (Pareto-based) approach, select and justify suitable optimization techniques, implement the selected methods, and evaluate the resulting solutions.

The project is not an algorithm-only exercise. The real-world problem, its mathematical formulation, constraints, competing objectives, algorithm selection, experimental evidence, and practical interpretation are all part of the project.

2. Core Requirement

Every project must contain the following:

- A clearly defined real-world problem.

- Decision variables that represent controllable decisions.
- At least two meaningful objectives that may involve trade-offs.
- Realistic constraints or assumptions.
- A mathematical formulation of the problem.
- A justified optimization approach.
- At least one baseline or reference approach for comparison.
- A multi-objective formulation and corresponding Pareto-based analysis of the competing objectives. If a group believes its problem is genuinely single-objective, it must say so in the topic proposal and obtain the professor's approval.
- Experimental evaluation using multiple runs or appropriate test conditions.
- A practical interpretation of the obtained solutions.

3. Important: Do Not Start With the Algorithm

Students must NOT begin by selecting an algorithm such as NSGA-II, PSO, GA, ACO, GWO, ABC, DE, SA, TLBO, etc. and then search for a problem to fit it.

The expected reasoning is:

Real-world problem → Decision variables → Objectives → Constraints → Problem classification → Algorithm selection → Implementation → Evaluation

The selected algorithm must be justified from the characteristics of the problem. Students must explain why a particular method is suitable and why simpler alternatives may or may not be sufficient.

4. Project Pathways

A group may follow one of the following pathways depending on its problem:

Path	Approach	Expected Work
A	Multi-objective	Formulate competing objectives and apply a suitable multi-objective optimization method.
B	Baseline + Multi-objective	Compare a simple/single-objective approach with a multi-objective approach.
C	Multi-objective + Hybrid	Identify a limitation and investigate a justified hybrid or improvement.
D	Multi-objective + Domain Heuristic	Integrate a domain-specific heuristic or improvement and evaluate its effect.

Hybridization is optional. It must not be added merely to make the project appear complex. A hybrid method should be introduced only when the students can identify a limitation of the primary method and provide experimental evidence for the proposed improvement.

4A. Research-Paper-Based Project Option — Four-Member Group

A group of exactly four students may choose a research paper that they find themselves as the starting point for its real-world optimization project. The paper is a starting point, not the final project.

- The project must still be completed by exactly four students and must satisfy the same 8-week project requirements.

- The selected paper must contain a genuine real-world optimization problem, or a problem that can be connected to a realistic decision-making application.
- Students must understand and reproduce the problem formulation before modifying or extending the work.
- Simply reproducing the paper, copying its code, or changing the dataset without a meaningful contribution is not sufficient.
- The group must clearly state what it will do beyond the original paper.
- The final project must include mathematical formulation, constraints, baseline/reference method, optimization implementation, multi-objective (Pareto) analysis, experiments, comparison and practical interpretation.
- All four members must make meaningful contributions and must be able to explain the paper, formulation, algorithms, experiments and results during the viva.

Acceptable Project Levels

- **Level 1 — Reimplementation + Analysis:** Reproduce the core optimization problem in a simplified but faithful form, validate the formulation, and analyze the obtained solutions. The group must still add meaningful analysis, such as sensitivity analysis, additional experiments or a justified comparison.
- **Level 2 — Comparative Study:** Reproduce the core problem and compare the paper's method with at least one other suitable optimization method or baseline.
- **Level 3 — Extension:** Extend the original problem by adding a justified objective, constraint, decision variable, dataset, uncertainty scenario, or realistic operating condition.
- **Level 4 — Hybrid / Improved Method:** Identify a limitation of the original approach and introduce a justified hybridization, local-search improvement, repair mechanism, adaptive strategy or other algorithmic improvement. The improvement must be evaluated experimentally.

Paths A–D (Section 4) describe the technical approach of any project. Levels 1–4 describe how far a paper-based project goes beyond its paper. A paper-based group states both its Level and its Path in the proposal.

What Counts as a Meaningful Contribution?

- Comparing the paper's algorithm with another course-relevant algorithm.
- Adding a genuine objective or constraint that changes the real decision problem.
- Testing the formulation on a different valid dataset or realistic scenario.
- Introducing uncertainty, robustness or sensitivity analysis.
- Developing a hybrid method and demonstrating whether it changes performance.
- Adding a domain-specific heuristic or constraint-repair mechanism.
- Studying scalability, convergence, runtime or Pareto-front quality under different problem sizes.

Important: "What is different from the original paper?" is a mandatory proposal question.

Research-Paper Proposal Format

- Research paper title and authors
- DOI / paper link
- Real-world problem addressed by the paper
- Decision variables
- Objectives and their practical meaning
- Constraints and assumptions

- Dataset / data source / simulation used in the paper
- Optimization algorithm used in the paper
- What the group will reproduce
- What the group will add/change beyond the paper
- Project Level (1–4) and Path (A–D)
- How the contribution will be evaluated
- Division of work among all four members
- Expected output and 8-week feasibility

IMPORTANT: The three papers listed below are ONLY SAMPLES to show students the kind of paper, problem and extension expected. They are illustrative, not a problem bank. NO group may select its project, paper or problem from this list, and topics that are the same as, or closely derived from, these samples will be rejected. Every group must discover its own problem and paper independently.

Paper / Domain	What the Paper Studies	Possible Four-Member Project Extension	DOI / Link
Building energy & thermal comfort	Multi-objective optimization for energy efficiency and thermal comfort in an existing office building using NSGA-II and fitness approximation.	Compare NSGA-II with another multi-objective method; add a realistic operating constraint or uncertainty scenario; analyze Pareto trade-offs.	10.1016/j.jobe.2021.102440 https://www.sciencedirect.com/science/article/pii/S2352710221002977
Vehicle routing with time windows	Multi-objective vehicle routing with time windows using NSGA-II and objective-specific variation operators.	Compare NSGA-II with another routing/metaheuristic approach; add fuel/emission or driver-work constraints; test different fleet sizes.	10.1016/j.eswa.2021.114779 https://www.sciencedirect.com/science/article/pii/S0957417421002207
ICU bed allocation	Two-stage multi-objective ICU bed allocation under multiple sources of uncertainty, using hospital data.	Create a smaller allocation model; compare multi-objective methods; add uncertainty/sensitivity experiments or fairness/resource constraints.	10.1038/s41598-023-45777-x https://www.nature.com/articles/s41598-023-45777-x

The above examples are samples only and are intentionally limited in number. They must not be used as project topics by any group.

Member	Primary Responsibility	Expected Evidence
Member 1	Paper study + problem formulation	Literature summary, decision variables, objectives, constraints, assumptions
Member 2	Baseline + original/paper method	Baseline implementation, reproduction of core method, validation experiments
Member 3	Second method / multi-objective / comparison	Alternative algorithm, Pareto analysis, evaluation metrics
Member 4	Extension / hybrid + experiments	Proposed contribution, hybrid or extension, robustness/scalability experiments

The above division is a suggestion. As described in Section 15, work must run in parallel and all four members must contribute every week.

If a research paper is used, students must find the paper themselves. The paper is a starting point for understanding and formulating the real-world problem; the four-member group must then reproduce, compare, extend or hybridize the work and demonstrate a contribution beyond the original paper.

5. Problem Discovery and Topic Selection Procedure

Students must find the project problem themselves. The instructor will not provide a predefined problem bank or a fixed list of research papers, and the sample papers in Section 4A must not be used as project topics. Each four-member group must identify THREE genuinely different real-world optimization problems and submit them for faculty approval.

The three candidates may be independently discovered real-world problems, research-paper-based problems, or a mixture of both. The three candidates must be meaningfully different; changing only the dataset, location or algorithm of the same underlying problem does not count as three different problems.

For every candidate, the group must identify the real decision-maker, the decision to be optimized, available or obtainable data, likely decision variables, at least two potentially competing objectives, and realistic constraints.

If a candidate is based on a research paper, the group must provide the paper title, authors, DOI or stable link, explain the real-world problem addressed, summarize the original formulation and method, and state what the group will do beyond the paper.

Students are responsible for checking that the required data, software, computational resources and implementation scope are feasible for a four-member group within eight weeks.

A suitable problem should involve an actual decision or allocation, measurable objectives, realistic constraints, and enough data or a defensible simulation plan for implementation and evaluation.

For literature-based discovery, students may search credible scholarly sources such as Google Scholar, IEEE Xplore, ScienceDirect, Springer, ACM Digital Library, PubMed and arXiv. For data discovery, students may use government/open-data portals, Kaggle, UCI, GitHub and other credible repositories.

Possible discovery sources include organizations, public services, campus/community activities, engineering or business processes, scientific applications, public datasets, open-source projects, and published research. These are discovery sources, not prescribed project topics.

The faculty member will review the three proposals and may approve one, ask the group to modify one, or ask the group to identify another problem. A topic that has already been approved for another group will not be approved again; the group will be asked to modify it or choose another.

Each of the three topics must meet the following conditions:

- The three topics should be sufficiently different and should not simply be different applications of the same algorithm.
- Each topic must be suitable for a four-member group and achievable within eight weeks.
- The proposed problem must contain a genuine optimization decision.
- The group must identify at least two potentially competing objectives.
- The group must identify realistic constraints and available data or a justified simulation.
- The group must mention possible baseline and optimization methods, but final algorithm selection may change after formulation.

6. Topic Proposal Format

Submit the following for each of the THREE proposed topics:

- Proposed project title
- Domain
- Real-world problem statement
- Who is affected by the problem / decision-maker
- Decision variables
- Objective 1
- Objective 2
- Optional Objective 3
- Constraints
- Available dataset / data source / simulation plan
- Why optimization is required
- Why the objectives may conflict
- Possible baseline method
- Possible optimization algorithms
- Possible hybridization or improvement (if applicable)
- Expected output / decision produced by the system

6A. After Topic Approval: Private Repository Submission

Once the professor has finalized and approved a group's topic, the group must submit a document containing its GitHub/GitLab repository link. The submission must contain:

- Group details: names and roll numbers of all four members.
- The final approved project title and the date of approval.
- The GitHub/GitLab repository link.
- The GitHub/GitLab usernames of the professor and of all four group members who have been given access.
- A confirmation that the repository visibility is set to Private.

Repository rules:

- The repository MUST be private. A public repository is not acceptable.
- Access must be given to the professor and to all other group members. All four members must be added as collaborators with permission to commit, since all members work on the same project and individual contributions are assessed from the commit history.
- The professor must be added before the repository link is submitted so that the repository can be opened and checked during the weekly reviews.
- The same repository must be used for the entire eight-week project. Do not create a new repository or change its link after submission without informing the professor.
- The repository link submission will be considered complete only when the repository is private and accessible to the professor and to all four members.

How to give access and where to submit:

- Professor's GitHub/GitLab username (add this account to your repository): [to be announced by the professor]
- The deadline and the method of submission for the repository-link document will be announced by the teacher.

- GitHub: open the repository, then Settings → Collaborators → Add people. GitLab: open the repository, then Manage/Settings → Members → Invite members. Give group members Write (GitHub) or Developer (GitLab) access; the professor needs at least Read access.

7. Required Mathematical Formulation

The final approved project must explicitly define:

- Decision variables: what the optimizer is allowed to change.
- Objective functions: what is minimized and/or maximized.
- Constraints: conditions that must be satisfied.
- Variable domains: continuous, discrete, binary, integer, or mixed.
- Assumptions: simplifications made because of data or implementation limitations.
- Problem type: linear/nonlinear, continuous/discrete/mixed, constrained/unconstrained, deterministic/stochastic, etc.

Students must distinguish between an objective and a constraint. For example, a maximum budget or minimum safety requirement should not automatically be treated as an optimization objective.

8. Multi-Objective Requirement

The project must demonstrate why multiple objectives are relevant. Students must discuss whether improving one objective can adversely affect another.

- Do not create artificial objectives merely to claim that the problem is multi-objective.
- Explain the practical meaning of each objective.
- Explain the trade-off between objectives.
- Generate and analyze non-dominated/Pareto solutions.
- Discuss why there is generally no single solution that is simultaneously best for all conflicting objectives.
- Select or recommend representative trade-off solutions only by explaining their characteristics; do not simply report one numerical optimum without discussing alternatives.

9. Algorithm Selection

There is no restriction on the choice of algorithm. Optimization is a process, and any suitable algorithm may be used, including algorithms that were not taught in the course, exact solvers, metaheuristics, problem-specific heuristics, learning-based methods, or a method designed by the group. The list below shows course techniques for reference only; it is not a limit on what may be used:

- Classical optimization: Linear Programming, Simplex, Duality and related methods.
- Unconstrained methods: Gradient Descent, Conjugate Gradient, Newton, BFGS and one-dimensional search methods.
- Constrained optimization: Lagrange multipliers, KKT, penalty, barrier and SQP approaches.
- Single-objective metaheuristics: Hill Climbing, Simulated Annealing, GA, DE, PSO, ABC, GWO, Bat Algorithm, TLBO and ACO.
- Multi-objective methods: NSGA-II and other justified Pareto-based or multi-objective evolutionary/swarm approaches.
- Hybrid methods when a clear limitation and improvement can be demonstrated.

The algorithm used in the final project must be selected after considering the mathematical structure, variables, constraints, objective functions, computational cost and characteristics of the real-world problem.

The only condition is that the final method must give better results than the baseline/reference approach (Section 10), and this must be demonstrated with evidence.

What "better" means:

- For a single objective, a better objective value, measured as the average over multiple independent runs.
- For multiple objectives, a better or more useful Pareto front: for example higher hypervolume, wider coverage of the trade-offs, or solutions that dominate those of the baseline.
- Lower runtime or cost also counts as an improvement when solution quality is comparable.
- Improvement must be shown on the metrics that suit the problem, over multiple runs where randomness is involved. A single lucky run is not evidence.

If the chosen method does not beat the baseline, analyze why, then change, tune or improve the method (or choose another one) until it does, and report what was tried. A hybrid or improved method (Section 4) is one way to do this.

If an existing library or solver is used, cite it in the README and describe how it was configured.

10. Baseline and Comparison

At least one baseline/reference approach is mandatory. Examples include:

- Random search
- Greedy or domain heuristic
- Local search / Hill Climbing
- A simple weighted-sum single-objective formulation
- An exact solver for a suitably small formulation
- An existing domain-specific method

The main optimization method must be compared against the baseline using appropriate metrics, and it must give better results than the baseline (see "What 'better' means" in Section 9), rather than only showing that the proposed method produces one feasible solution.

11. Evaluation Requirements

- Use multiple independent runs where randomness is involved.
- Report objective values of obtained solutions.
- Show convergence behaviour where applicable.
- Show the Pareto front or non-dominated solution set for multi-objective projects.
- Report appropriate quality indicators such as hypervolume, spacing, number of non-dominated solutions, runtime, or problem-specific measures where applicable.
- Report constraint violations or feasibility information.
- Show, with tables or plots, that the main method gives better results than the baseline/reference approach.
- Interpret the results in terms of the original real-world problem.

12. GitHub / GitLab Requirement

Every group must maintain the project continuously on GitHub or GitLab. The repository will be checked every week. Uploading the complete project near the deadline will not be considered a meaningful weekly development process.

The repository must be PRIVATE and shared with the professor and all four group members, as described in Section 6A. The programming language, code organization, folder structure and README format must follow Section 12A.

Recommended repository structure:

```
project/
├── README.md
├── .gitignore
├── problem/              (approved topic, problem statement)
├── literature/           (papers, notes, summaries)
├── formulation/          (mathematical formulation)
├── data/
├── src/                  (all source code; see language examples in Section 12A)
│   ├── baseline/
│   ├── algorithms/
│   ├── multiobjective/
│   └── hybrid/
├── experiments/          (configurations, experiment run scripts)
├── results/              (raw outputs: CSV, JSON, logs)
├── figures/              (plots, Pareto fronts)
├── report/
└── presentation/
```

This is the standard layout for every project, in every language. Baseline, algorithm, multi-objective and hybrid code must stay in separate, clearly named files or folders. The language examples in Section 12A are this same layout with the entry and build files added; the folders problem/, literature/, formulation/, report/ and presentation/ are still required.

12A. Programming Language, Code Organization and README Standards

Groups may use any programming language that suits the problem. Whatever the language, the project must be systematic, easy to download, easy to run, and easy to read. During the weekly checks and the viva, the professor may clone the repository on a fresh machine and follow the README. A project that cannot be built and run by following its README is treated as incomplete.

General rules for every language:

- One clearly named entry point (for example main.cpp, main.py or Main.java) that runs the project with a single documented command.
- Separate files for each part: problem definition (objective functions and constraints), baseline, each optimization algorithm, multi-objective components, hybrid/improved method, evaluation metrics, visualization and the experiment runner. Do not put the whole project in one file or in one notebook.
- Algorithms and hybrid approaches must be in their own clearly named files or folders (for example algorithms/nsga2.* and hybrid/hybrid_method.*) so any algorithm can be found within seconds.

- Results and visualizations must be kept in separate folders, not mixed with source code: results/ for raw outputs (CSV, JSON, logs) and figures/ for plots (Pareto fronts, convergence curves, comparison charts). These must be generated by the project code, not made manually.
- Random seeds, parameters and the number of runs must be set in a configuration file or command-line arguments and documented, so results can be reproduced.
- All dependencies must be listed in a file (requirements.txt, Makefile, pom.xml, package.json, Project.toml, etc.) with versions where possible. Use relative paths only; no absolute paths such as C:\Users\...
- Use meaningful, consistent file and folder names (lowercase, no spaces; for example nsga2.py, not final2.py) and short comments or docstrings explaining each module.
- Add a .gitignore. Do not commit compiled files (.exe, .o), virtual environments, __pycache__, node_modules, IDE settings, large datasets, passwords or API keys.
- Small datasets go in data/ with a short description of the source, whether the data is real or synthetic, and its fields. Large datasets must not be committed; provide a download link or script and instructions.
- Before the final submission, test the project by cloning the repository into a new folder and following the README exactly.

If the project is in C++:

- A Makefile in the repository root is compulsory. At minimum: make builds the project, make run runs it, and make clean removes build output.
- The README must state the compiler and standard (for example g++ with C++17) and the operating system on which the project was tested. On Windows, use WSL or MinGW/MSYS2 so that make works, and say which one in the README.
- A main file (main.cpp) must be present. It should only act as the driver (read configuration, call the modules, write results); algorithm code must not be written inside main.cpp.
- Each algorithm, the baseline, the problem definition, the metrics and each hybrid/improved method must be in separate .cpp/.h file pairs.
- Compiled binaries and object files go to a build/ (or bin/) folder that is listed in .gitignore.
- C++ programs should write results to results/ (for example as CSV). If plots are made with a separate script (Python, gnuplot, etc.), place the script in scripts/, save the plots in figures/ and document the command in the README.

Example C++ layout:

```
project/
├── README.md
├── Makefile
├── .gitignore
├── problem/
├── literature/
├── formulation/
├── data/
├── src/
│   ├── main.cpp                    (driver only: reads config, calls modules)
│   ├── problem.cpp / problem.h     (objectives, constraints)
│   ├── metrics.cpp / metrics.h
│   ├── baseline/
│   │   └── baseline.cpp / baseline.h
│   ├── algorithms/
│   │   └── nsga2.cpp / nsga2.h     (one file pair per algorithm)
│   ├── multiobjective/             (non-dominated sorting, etc.)
```

```
│   └── hybrid/
│       └── hybrid_method.cpp / hybrid_method.h
├── build/                      (git-ignored: binaries, object files)
├── experiments/
├── scripts/                    (optional plotting scripts)
├── results/                    (raw outputs: CSV, logs)
├── figures/                    (plots, Pareto fronts)
├── report/
└── presentation/
```

If the project is in Python:

- The project must use several files/modules. A single .py file, or a project that exists only as a Jupyter notebook, is not acceptable. Notebooks may be used for exploration or presentation, but the same code must also run from scripts.
- Provide main.py (or an equivalent entry script) plus separate modules for the problem definition, baseline, each algorithm (algorithms/ folder), hybrid method (hybrid/ folder), metrics and visualization.
- Provide requirements.txt (or environment.yml / pyproject.toml) and state the Python version. The README must show how to create a virtual environment and install the dependencies.
- Plots must be generated by a visualization module or script and saved in figures/; raw results must be saved in results/.

Example Python layout:

```
project/
├── README.md
├── requirements.txt
├── .gitignore
├── main.py                     (entry point)
├── problem/
├── literature/
├── formulation/
├── data/
├── src/
│   ├── problem.py              (objectives, constraints)
│   ├── metrics.py
│   ├── visualization.py
│   ├── baseline/
│   │   └── baseline.py
│   ├── algorithms/
│   │   └── nsga2.py            (one file per algorithm)
│   ├── multiobjective/
│   └── hybrid/
│       └── hybrid_method.py
├── experiments/                (run_experiments.py, configs)
├── notebooks/                  (optional, exploration only)
├── results/                    (raw outputs: CSV, logs)
├── figures/                    (plots, Pareto fronts)
├── report/
└── presentation/
```

If the project is in any other language (Java, MATLAB, R, Julia, JavaScript, etc.):

- Use the standard build and dependency tool of that language (for example Maven/Gradle, Project.toml, renv or DESCRIPTION, package.json) and include its file in the repository.

- The README must give the exact commands to install, build and run the project, and what output to expect. Someone new to the project must be able to download and run it without asking the group.
- The same rules apply: an entry/main file, separate files for the baseline, algorithms and hybrid approaches, and separate results/ and figures/ folders.
- Prefer free and open-source tools. If a language or tool needs a paid or licensed product (for example MATLAB and its toolboxes), discuss it with the professor in advance and state the exact version in the README.

If the project has a frontend or user interface:

- This applies to web applications, dashboards and desktop GUIs (for example Streamlit, Flask/Django with a web page, React or other JavaScript frameworks, Tkinter).
- Keep the frontend in its own folder (for example frontend/) and the optimization code in a separate folder (for example backend/ or src/). The optimization code must also be runnable from the command line without the frontend.
- The frontend must have its own dependency file (package.json, requirements.txt, etc.). The README must explain how to install it, the exact command to start it, the address/port to open, and how it connects to the backend.
- Do not commit node_modules/, build or dist folders, or .env files. If configuration is needed, provide a .env.example with placeholder values.
- Include screenshots of the interface in the repository (for example in figures/) and show them in the README.
- A frontend does not replace the required experiments. Baseline comparison, multiple runs, Pareto analysis and metrics must still be produced by the project code and reported.

Example layout for a project with a frontend:

```
project/
├── README.md
├── backend/                (optimization code + own dependency file)
│   ├── main.py
│   ├── algorithms/
│   └── requirements.txt
├── frontend/               (UI code + own dependency file)
│   ├── package.json
│   └── src/
├── data/
├── results/
├── figures/                (include UI screenshots)
└── report/
```

README.md standard (GitHub format):

Every repository must have a README.md in the root folder, written in Markdown so that GitHub/GitLab shows it on the repository home page. It must contain the following sections, in this order:

- Project title and a one-line description.
- Group details: group number, names, roll numbers and GitHub/GitLab usernames of all four members, and the professor's name.
- Problem statement: the approved topic, the decision-maker and why optimization is required.
- Formulation summary: decision variables, objectives, constraints and problem type (the full formulation is in formulation/).

- Methods implemented: the baseline, main algorithm(s) and hybrid/improved method, with the file or folder location of each.
- Repository structure: the folder tree with a one-line description of each main folder.
- Requirements: operating system, language version, libraries and tools.
- Installation: exact commands to clone, install dependencies and build.
- How to run: exact commands to run the main program, reproduce all experiments and generate the figures.
- Data: source, whether real or synthetic, how to obtain it, and a description of the fields.
- Results: a short summary with the main tables and figures (Pareto front, comparison with the baseline, metrics) shown from the figures/ folder.
- Individual contributions of each member.
- Weekly progress log.
- References: papers or sources used, with title, authors and DOI/link.

README formatting rules:

- Use Markdown headings (#, ##), tables for results and members, and code blocks (triple backticks) for every command so that students can copy and run them.
- Show figures using relative image links (for example ![Pareto front](figures/pareto_front.png)). Do not use absolute local paths or screenshots of code.
- The README must be kept up to date. Update it whenever the folder structure or the run commands change, and before each weekly check.
- A README that contains only the project title, or that has commands which do not work, is not acceptable.
- A ready-to-use README template is given in Appendix A at the end of this document and is also provided as README_TEMPLATE.md.

13. Weekly Development Plan

Week	Focus	Minimum Repository Update	Verification
1	Problem identification	3 topic proposals and literature survey (kept in a shared document until approval); after approval: private repository created, initial README, repository-link document submitted	Problem relevance and feasibility
2	Problem formulation	Variables, objectives, constraints, assumptions, data plan	Mathematical formulation
3	Baseline	Baseline implementation and first results	Working baseline
4	Optimization method	First selected optimization algorithm + experiments	Algorithm and reasoning
5	Multi-objective optimization	MO formulation, Pareto-based method, initial front	MO formulation and implementation
6	Comparison / improvement	Comparative experiments; optional hybrid/improvement	Evidence-based improvement
7	Evaluation	Multiple runs, metrics, plots, analysis	Results and interpretation
8	Finalization	Final code, report, figures, README, presentation	Final repository and demo

14. Meaningful Git Commits

Weekly commits must reflect actual development. Examples of meaningful commits:

- Added mathematical formulation and constraint definitions.
- Implemented baseline scheduler.
- Added constraint-repair mechanism.
- Implemented non-dominated sorting.
- Added Pareto-front visualization.
- Added hypervolume calculation.
- Compared three random seeds / independent runs.
- Implemented local-search improvement.
- Added experiment configuration and result logging.

Avoid meaningless commit messages such as:

- final
- final2
- updated
- changes
- done
- project

15. Division of Work — Four-Member Group

The group should divide responsibilities, but every member must understand the complete project and be able to explain the mathematical model, algorithms, results and conclusions.

The work must be carried out in parallel, not one member after another. All four members must contribute every week from Week 1 to Week 8, and no member should wait for another member to finish before starting. The role table and the week-by-week plan below are SUGGESTIONS; a group may adapt them to its own problem, but every member must still make meaningful, visible contributions in every week.

Member	Suggested Primary Responsibility	Typical Deliverables
Member 1	Problem & mathematical model	Problem definition, variables, objectives, constraints
Member 2	Baseline & optimization implementation	Baseline, algorithm implementation, code
Member 3	Multi-objective optimization	Pareto method, non-dominated solutions, MO analysis
Member 4	Data, experiments & evaluation	Data pipeline, experiments, metrics, visualization

Suggested Week-by-Week Parallel Work Plan

Week	Member 1 (Problem & model)	Member 2 (Baseline & algorithm)	Member 3 (Multi-objective)	Member 4 (Data, experiments & evaluation)
1 — Problem identification	Write problem statements, decision-makers and decisions for the 3 topics	Search papers / literature for the 3 candidates; start literature survey	List possible objectives, conflicts, baselines and algorithms per topic	Check data availability or simulation plan; after approval, create the private repository, add the professor and all members, and start the README

Week	Member 1 (Problem & model)	Member 2 (Baseline & algorithm)	Member 3 (Multi-objective)	Member 4 (Data, experiments & evaluation)
2 — Formulation	Define decision variables, objectives, constraints and assumptions	Classify the problem type; design the baseline approach	Draft multi-objective formulation and explain the objective conflict	Collect / clean data or build simulator; set up repository structure
3 — Baseline	Check that the formulation matches what the baseline computes	Implement the baseline	Implement objective functions and constraint / feasibility checker	Write experiment scripts and logging; produce first baseline results
4 — Optimization method	Justify algorithm choice from the problem structure	Implement the selected algorithm	Implement non-dominated sorting and other multi-objective components	Run first experiments; plot convergence
5 — Multi-objective	Interpret the meaning of each trade-off in the real problem	Add constraint handling / repair to the algorithm	Generate the Pareto front with the multi-objective method	Implement hypervolume, spacing and other metrics; make Pareto plots
6 — Comparison / improvement	State the limitation and the reason for any improvement	Implement the improvement / hybrid (if used)	Implement the second method or weighted-sum comparison	Run comparative experiments with multiple seeds
7 — Evaluation	Write the practical interpretation of the solutions	Run sensitivity and scalability tests	Analyse Pareto solutions; select representative trade-off solutions	Run final multiple runs; prepare result tables and figures
8 — Finalization	Report sections: problem, formulation, limitations	Clean code; write execution instructions in README	Report sections: multi-objective analysis; prepare presentation	Final figures, demo and repository check

Weekly contribution rules:

- Every member must make at least one meaningful commit under his or her own GitHub/GitLab account every week. Commits made by one person on behalf of others do not count as the other members' contributions.
- Members may overlap, review each other's work and swap tasks, but no member may be left without a task in any week.
- Each member must record his or her weekly contribution in the repository (for example in a short weekly log in the README or report folder).
- Weekly repository checks will look at each member's individual commits. A member with no meaningful contribution in a week may be questioned about it and assessed accordingly.

16. Expected Final Deliverables

- GitHub/GitLab repository with complete commit history (private, shared with the professor and all four members).
- Final mathematical formulation.
- Source code and execution instructions.
- Dataset or simulation description.
- Baseline implementation.
- Selected optimization method(s).
- Multi-objective implementation and Pareto analysis.
- Experimental results and comparison.
- Plots/tables supporting the conclusions.
- Final project report.
- Final presentation/demo.

17. Final Report Structure

1. Introduction and real-world motivation
2. Problem statement
3. Literature review / existing approaches
4. Decision variables
5. Objective functions
6. Constraints and assumptions
7. Problem classification
8. Baseline approach
9. Optimization algorithm selection and justification
10. Multi-objective formulation
11. Algorithm implementation
12. Experimental setup
13. Results and Pareto analysis
14. Comparison and discussion
15. Practical interpretation
16. Limitations
17. Future work
18. Individual contributions
19. GitHub/GitLab repository link

18. Project Assessment Criteria

Component	What will be assessed	Suggested Weight*
Problem selection & real-world relevance	Importance, clarity, feasibility and genuine optimization need	10%
Mathematical formulation	Variables, objectives, constraints, assumptions and classification	15%
Algorithm selection	Technical justification and connection to problem characteristics	10%
Implementation	Correctness, code organization, repository/README format and reproducibility	15%
Multi-objective analysis	Pareto solutions, trade-offs and appropriate metrics	15%
Experiments & comparison	Baselines, repeated runs, improvement over the baseline, convergence/quality/runtime analysis	15%
GitHub/GitLab weekly development	Meaningful weekly progress and individual contributions	10%
Report, presentation & viva	Clarity, interpretation and ability to defend the work	10%

*Suggested project assessment distribution. The final marks and split between theory and lab will be as prescribed by the course instructor.

19. Important Rules

- The project must solve a real decision problem, not merely demonstrate an optimization algorithm.
- The professor will approve the final topic.
- Research-paper-based projects are also four-member group projects and must satisfy all project requirements; a paper alone does not count as the project contribution.
- Each group must submit three possible topics before approval.

- The sample papers given in this document are illustrations only. No group may take its project from the sample list.
- After the topic is approved, the group must submit a document with its GitHub/GitLab repository link. The repository must be private, with access given to the professor and all four group members.
- The same topic/problem formulation should not be duplicated across groups unless specifically approved.
- Use of publicly available datasets, APIs, simulators or appropriately generated synthetic data is permitted, but the source and assumptions must be documented.
- Do not fabricate real-world results. Clearly distinguish real data from simulated or synthetic data.
- Algorithm selection must be justified.
- Any algorithm may be used, including algorithms not covered in the course. The final method must give better results than the baseline, supported by evidence.
- A hybrid algorithm is optional and must be justified by evidence.
- Weekly GitHub/GitLab updates are compulsory.
- The code and repository must follow Section 12A (Makefile and main file for C++, multiple modules for Python, separate results/ and figures/ folders, a complete README.md). The project must be easy to download and run from the README.
- Late bulk uploading of work will not count as weekly progress.
- Each student must make meaningful contributions and must be able to explain the complete project during the viva.
- The work must be divided so that all four members contribute in parallel every week; sequential hand-offs where some members are idle are not acceptable.
- The final project must report trade-offs and not hide alternative Pareto solutions simply to present one preferred answer.

20. Student Submission Checklist

- ☐ Group contains exactly four students.
- ☐ Three project topics submitted.
- ☐ Final topic approved by faculty.
- ☐ Topic was not taken from the sample list given in this document.
- ☐ Document with the repository link submitted after topic approval.
- ☐ Repository is private and access is given to the professor and all four group members.
- ☐ Problem was independently identified and clearly defined.
- ☐ Decision variables identified.
- ☐ At least two meaningful objectives identified.
- ☐ Constraints identified.
- ☐ Mathematical formulation completed.
- ☐ Baseline implemented.
- ☐ Optimization algorithm selected and justified.
- ☐ Main method shown to give better results than the baseline, with evidence over multiple runs.
- ☐ Multi-objective approach and Pareto analysis implemented (or a professor-approved exception).
- ☐ Pareto/non-dominated solutions analyzed.
- ☐ Experiments repeated where randomness is involved.
- ☐ Appropriate evaluation metrics reported.
- ☐ GitHub/GitLab updated every week.

- ☐ Code is organized as per Section 12A (separate files for baseline, algorithms and hybrid; entry/main file; Makefile if C++).
- ☐ Results and figures are stored in separate results/ and figures/ folders.
- ☐ README.md follows the required format and the project runs from a fresh clone by following it.
- ☐ Every member has made meaningful commits under his or her own account every week.
- ☐ Individual contributions documented.
- ☐ If using a research paper, the paper was independently identified, cited, and the four-member group contribution beyond the original paper is clearly documented.
- ☐ Final report and presentation completed.

21. Final Guidance to Students

Find a problem where an actual decision has to be made and where different desirable outcomes can conflict. Do not select a topic merely because an algorithm is easy to code. Start from the real-world problem, collect evidence about why it matters, formulate the decision variables/objectives/constraints, and only then select suitable optimization methods.

Appendix A. README.md Template

Copy the text below into README.md in the root of your repository and replace every item in [square brackets]. If a section does not apply, write “Not applicable” and explain why.

```
# Project Title

[One or two lines describing the real-world optimization problem and the decision it supports.]

## 1. Group Details

| Member | Roll No. | GitHub/GitLab Username |
|--------|----------|------------------------|
| Member 1 | [roll no.] | [username] |
| Member 2 | [roll no.] | [username] |
| Member 3 | [roll no.] | [username] |
| Member 4 | [roll no.] | [username] |

- Course: Optimization Techniques
- Professor: [name]
- Group number: [number]
- Approved topic date: [date]

## 2. Problem Statement

[Real-world problem, who the decision-maker is, what decision is being optimized, and why optimization
is required.]

## 3. Formulation Summary

- **Decision variables:** [what the optimizer can change, with domains]
- **Objectives:** [Objective 1, Objective 2, ...] and why they conflict
- **Constraints:** [main constraints]
- **Problem type:** [linear/nonlinear, continuous/discrete/mixed, deterministic/stochastic, ...]

The full mathematical formulation is in `formulation/`.

## 4. Methods Implemented

| Method | Purpose | Location |
|--------|---------|----------|
| [Baseline, e.g. random search] | Reference for comparison | `baseline/` |
| [Main algorithm, e.g. NSGA-II] | Multi-objective optimization | `algorithms/` |
| [Hybrid / improvement, if any] | [What limitation it addresses] | `hybrid/` |

## 5. Repository Structure

```
project/
├── README.md
├── ...
```

[One-line description of each main folder.]

6. Requirements
```

- Operating system: [e.g. Ubuntu 22.04 / Windows 11 with WSL]
- Language and version: [e.g. Python 3.11 / g++ 11 with C++17]
- Libraries / tools: [listed in `requirements.txt` / `Makefile` / `package.json`]

7. Installation

```bash
git clone [repository link]
cd [repository folder]
[install / build commands, e.g. pip install -r requirements.txt   or   make]
```

8. How to Run

Run the main program:

```bash
[command, e.g. python main.py   or   make run]
```

Reproduce all experiments (multiple runs, fixed seeds):

```bash
[command]
```

Generate the figures:

```bash
[command]
```

Outputs are written to `results/` (raw data, logs) and `figures/` (plots).

9. Data

- Source: [link / organization / simulator]
- Real or synthetic: [state clearly]
- How to obtain: [included in `data/` or download script/link]
- Description: [columns / fields and assumptions]

10. Results

[Short summary of the main findings: Pareto front, comparison with the baseline, quality indicators, runtime.]



Method	Metric 1	Metric 2	Runtime
Baseline			
Main method			

11. Individual Contributions

Member	Contribution

Member 1	[what this member did]
Member 2	[what this member did]
Member 3	[what this member did]
Member 4	[what this member did]

12. Weekly Progress Log

Week	Work done	Members involved
1		
2		

13. References

1. [Author(s), Title, Journal/Conference, Year, DOI/link]